

Nome e cognome: Giuseppe Carlino;

Progetto: ChatLab.

Relazione per “ChatLabServer”

Per la strutturazione del server si è scelto di usufruire dell’interfaccia grafica offerta dal package swing, facendo puro ausilio del motore grafico di NetBeans (v. 20).



Di default, il server è settato sull’indirizzo 192.168.153.1 ed è in ascolto sulla porta 12345.

```
// Dichiarazione ed implementazione variabili
int porta = 12345;

// Dichiarazione ed implementazione oggetti
ServerFrame interfacciaGrafica = new ServerFrame();
InetAddress indirizzo = null;

// Impostazione dell'indirizzo IP dell'host
try {
    indirizzo = InetAddress.getByName("192.168.153.1");
} catch (UnknownHostException e) {

    // Visualizzare un pop-up di errore (host sconosciuto)
    showMessageDialog(null, "Host sconosciuto: " + e.getMessage() + '.', "Errore", ERROR_MESSAGE);
}

// Verificare che la comunicazione client-server vada sempre a buon fine
try (ServerSocket serverSocket = new ServerSocket(porta, 0, indirizzo)) {

    // Messaggio di conferma di ascolto (con IP dato) sulla porta indicata
    interfacciaGrafica.setAreaOutputServer("Server in ascolto su <" + indirizzo + ':' + porta + ">.");
```

Nome e cognome: Giuseppe Carlino;

Progetto: ChatLab.

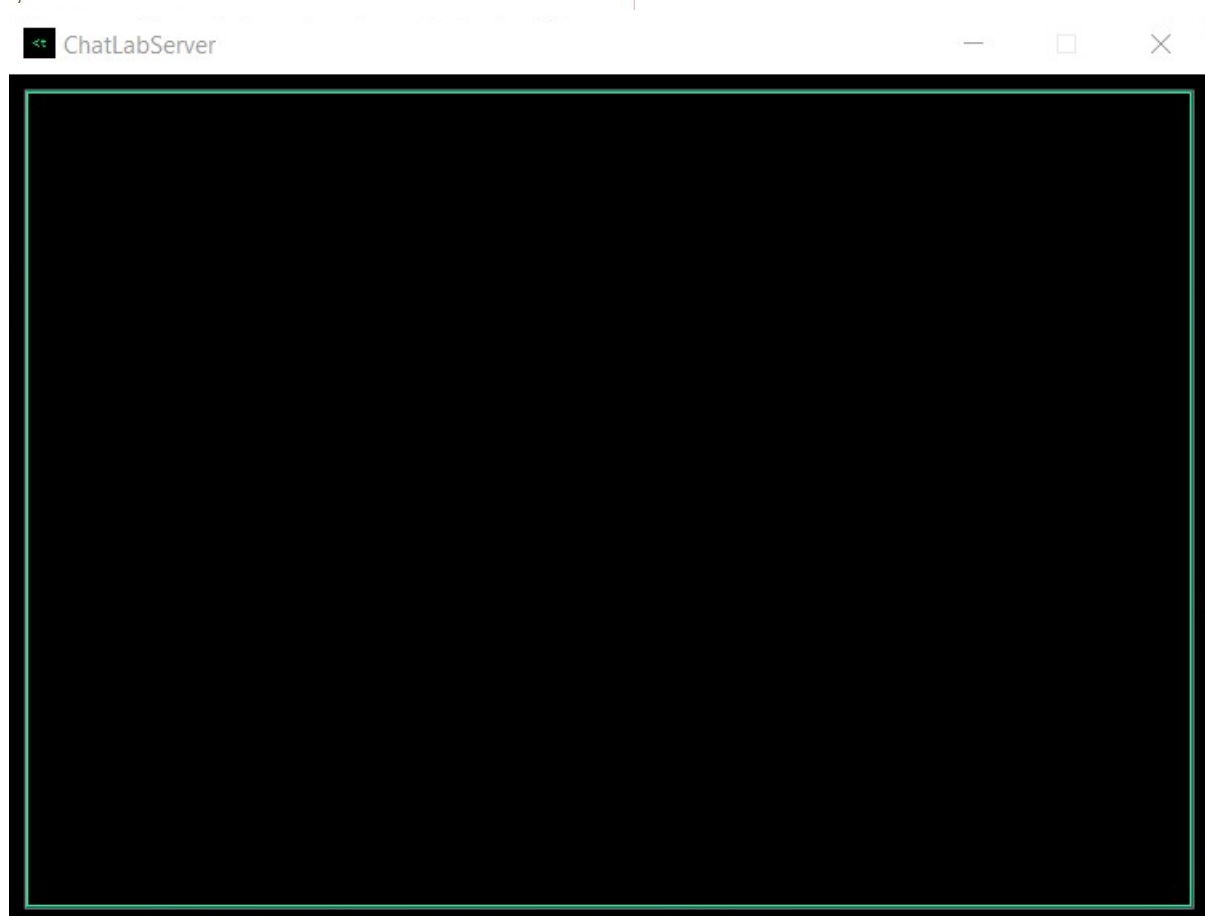
Il server accetta richieste di connessione fino a quando non si decide di chiudere la relativa finestra di interfaccia grafica. Ogni qualvolta un client richieda di connettersi alla rete, per quest'ultimo viene istanziato un gestore, appartenente alla classe ClientHandler, il quale lo memorizzerà anche in una lista di client connessi a quella rete (nickname + indirizzo IP).

```
// Concedere infinite richieste di connessione
while (true) {

    // Accettazione di una richiesta di connessione
    Socket clientSocket = serverSocket.accept();

    // Messaggio di conferma di avvenuta connessione
    interfacciaGrafica.setAreaOutputServer("\nConnessione accettata da " + clientSocket.getInetAddress() + ".\n");

    // Avviare la comunicazione
    ClientHandler client = new ClientHandler(interfacciaGrafica, clientSocket);
    client.start();
}
```



```
public class ClientHandler extends Thread {

    // Dichiarazione attributi
    private final ServerFrame interfacciaGrafica;
    private final Socket clientSocket;
    private String nickname;

    // Dichiarazione attributi statici
    public static ArrayList<ClientHandler> listaClient = new ArrayList<>();

    // Metodi costruttori
    public ClientHandler(ServerFrame interfacciaGrafica, Socket clientSocket) {
        this.interfacciaGrafica = interfacciaGrafica;
        this.clientSocket = clientSocket;
        nickname = "";
    }
}
```

Nome e cognome: Giuseppe Carlino;

Progetto: ChatLab.

Per l'autenticazione, il client deve aver inviato un messaggio secondo la sintassi NICK <Nickname>. La proposta di autenticazione viene inviata nuovamente ogni qualvolta il client ha errato, magari dimenticandosi del comando NICK, o inserendo un nickname già esistente, o semplicemente inviando tutt'altro.

```
// Richiesta di autenticazione da parte del server per il client connesso
do {

    // Resetare il valore di isEsistente su false
    isEsistente = false;

    // Lettura del messaggio ricevuto
    messaggioClient = messaggioClientIn.readLine();

    // Stampa del messaggio ricevuto
    interfacciaGrafica.setAreaOutputServer("\nNuovo Utente: " + messaggioClient);

    // Se il messaggio ricevuto contiene "NICK "
    if (messaggioClient.contains("NICK ")) {

        // Frammentazione di messaggioClient in due frammenti, dei quali si utilizza l'ultimo: nickname
        messaggioClientFrammenti = messaggioClient.split(" ", 2);

        // Estrazione del nickname
        nickname = messaggioClientFrammenti[1];

        // Verifica dell'esistenza del nuovo client
        while (!isEsistente && i < listaClient.size()) {
            client = listaClient.get(i);
            if (client.getNickname().equals(nickname))
                isEsistente = true;
            i++;
        }
    }
}
```

Se il client si è autenticato, il server invia un messaggio di benvenuto al client (identificandosi con il nome di App), avvisando anche gli altri utenti della rete del suo arrivo; altrimenti, trasmette un messaggio di errore, diverso a seconda della tipologia di errore commesso. Inoltre, fin quando il client non si è autenticato, quest'ultimo viene identificato come Nuovo Utente (come visualizzato nel codice in alto).

```
// Se il nuovo client esiste già
if (isEsistente) {

    // Inviare un messaggio di errore
    messaggioServerOut.println("App: Attenzione! Utente già esistente.");
}

// Altrimenti
} else {

    // Inviare un messaggio di errore
    messaggioServerOut.println("App: Attenzione! Seguire la seguente sintassi: NICK <Nickname>!");
}

} while (!messaggioClient.contains("NICK ") || isEsistente);

// Aggiunta del nuovo client alla lista di client connessi alla rete di comunicazione
listaClient.add(this);

// Messaggio di segnalazione di autenticazione avvenuta con successo indirizzato al nuovo client autenticato
messaggioServerOut.println("App: Ok! Benvenuto " + nickname + "!");

// Messaggio di segnalazione di autenticazione avvenuta con successo indirizzato a tutti gli altri client della rete di comunicazione
for (ClientHandler c : listaClient) {
    if (!c.getNickname().equals(nickname)) {
        messaggioServerOut = new PrintWriter(c.getClientSocket().getOutputStream(), true);
        messaggioServerOut.println("App: " + nickname + " ha effettuato l'accesso.");
    }
}
```

Nome e cognome: Giuseppe Carlino;

Progetto: ChatLab.

Ecco elencate tutte le modalità di comunicazione che il server è in grado di gestire (tramite il gestore ClientHandler):

- **CHAT <Nickname>**: il server stabilisce una comunicazione privata tra il client richiedente e il client indicato dal nickname digitato dal richiedente. D'ora in poi, al client [richiedente] occorre semplicemente digitare il messaggio (il <Corpo>) da trasmettere, e questo viene automaticamente inviato al client digitato precedentemente, tramite l'intermezzo del server. Per terminare tale comunicazione, il client deve digitare STOP;

```
// Comunicazione Unicast
if (messaggioClient.contains("CHAT ")) {

    // Implementazione variabili (locali)
    i = 0;
    isEsistente = false;

    // Implementazione oggetti (locali)
    client = null;

    // Frammentazione di messaggioClient in due frammenti, dei quali si utilizza l'ultimo: nicknameDestinatario
    messaggioClientFrammenti = messaggioClient.split(" ", 2);

    // Assegnazione dei frammenti alle relative locazioni di memoria
    nicknameDestinatario = messaggioClientFrammenti[1];

    // Verifica dell'esistenza del client destinatario
    while (!isEsistente && i < listaClient.size()) {
        client = listaClient.get(i);
        if (client.getNickname().equals(nicknameDestinatario))
            isEsistente = true;
        i++;
    }

    // Avviare la comunicazione Unicast
    messaggioServerOut.println("App: Comunicazione con " + nicknameDestinatario + " abilitata!");
    do {
        messaggioClient = messaggioClientIn.readLine();
        if (!messaggioClient.equals("STOP")) {
            messaggioServerOut = new PrintWriter(client.getClientSocket().getOutputStream(), true);
            messaggioServerOut.println(nickname + ": " + messaggioClient);
            messaggioServerOut = new PrintWriter(clientSocket.getOutputStream(), true);
            messaggioServerOut.println("App: Messaggio inviato!");
        }
    } while (!messaggioClient.equals("STOP"));

    // Messaggio di avvenuta terminazione della comunicazione Unicast
    messaggioServerOut.println("App: Comunicazione con " + nicknameDestinatario + " disabilitata!");
```

Nome e cognome: Giuseppe Carlino;

Progetto: ChatLab.

- **BROADCAST <Corpo>**: il server trasmette il messaggio del client a tutti gli altri utenti della rete;

```
// Comunicazione Broadcast
} else if (messaggioClient.contains("BROADCAST ")) {

    // Frammentazione di messaggioClient in due frammenti, dei quali si utilizza l'ultimo: corpoMessaggio
    messaggioClientFrammenti = messaggioClient.split(" ", 2);

    // Assegnazione dei frammenti alle relative locazioni di memoria
    corpoMessaggio = messaggioClientFrammenti[1];

    // Invio della risposta a tutti i client connessi alla rete di comunicazione
    for (ClientHandler c : listaClient) {
        if (!c.getNickname().equals(nickname)) {
            messaggioServerOut = new PrintWriter(c.getClientSocket().getOutputStream(), true);
            messaggioServerOut.println(nickname + ": " + corpoMessaggio);
        }
    }

    // Riconessione al client mittente
    messaggioServerOut = new PrintWriter(clientSocket.getOutputStream(), true);
    messaggioServerOut.println("App: Messaggio inviato!");
}
```

- **NICK <Nickname>**: il server concede il permesso al client di rinominarsi, verificando sempre che il nuovo nickname sia diverso da tutti quelli già esistenti, appartenenti agli altri client in rete;

```
// Cambio nickname
} else if (messaggioClient.contains("NICK ")) {

    // Dichiarazione ed implementazione variabili (locali)
    String vecchioNickname;

    // Implementazione variabili (locali)
    i = 0;
    isEsistente = false;

    // Frammentazione di messaggioClient in due frammenti, dei quali si utilizza l'ultimo: nickname
    messaggioClientFrammenti = messaggioClient.split(" ", 2);

    // Verifica dell'esistenza del nickname impostato dal client
    while (!isEsistente && i < listaClient.size()) {
        client = listaClient.get(i);
        if (client.getNickname().equals(messaggioClientFrammenti[1]))
            isEsistente = true;
        i++;
    }

    // Rimozione del vecchio client dalla lista di client connessi alla rete di comunicazione
    listaClient.remove(this);

    // Inserimento del nuovo nickname
    nickname = messaggioClientFrammenti[1];

    // Aggiunta del nuovo client alla lista di client connessi alla rete di comunicazione
    listaClient.add(this);

    // Invio della risposta del server
    messaggioServerOut.println("App: Modifica effettuata con successo!");

    // Invio del messaggio di avvenuta modifica del nickname del client a tutti i client connessi alla rete di comunicazione
    for (ClientHandler c : listaClient) {
        if (!c.getNickname().equals(nickname)) {
            messaggioServerOut = new PrintWriter(c.getClientSocket().getOutputStream(), true);
            messaggioServerOut.println("App: " + vecchioNickname + " ha modificato il proprio nickname in " + nickname + '.');
        }
    }

    // Riconessione al client che ha modificato il proprio nome
    messaggioServerOut = new PrintWriter(clientSocket.getOutputStream(), true);
}
```


Nome e cognome: Giuseppe Carlino;

Progetto: ChatLab.

- **LIST:** il server trasmette al client richiedente una lista di tutti i client connessi alla rete di comunicazione, chiamata listaClient;

```
// Lista di client connessi alla rete di comunicazione
} else if (messaggioClient.equals("LIST")) {

    // Invio della lista di nickname dei client connessi alla rete di comunicazione
    messaggioServerOut.println(Arrays.toString(getListaClientNickname()));
```

- **QUIT:** il server elimina il client da listaClient, prima di effettuare la disconnessione vera e propria.

```
// "Disconnessione" del client dalla rete di comunicazione
} else if (messaggioClient.equals("QUIT")) {

    // Rimozione del client dalla lista di client connessi alla rete di comunicazione
    listaClient.remove(this);

    // Invio del messaggio di avvenuta "disconnessione" del client a tutti i client connessi alla rete di comunicazione
    for (ClientHandler c : listaClient) {
        if (!c.getNickname().equals(nickname)) {
            messaggioServerOut = new PrintWriter(c.getClientSocket().getOutputStream(), true);
            messaggioServerOut.println("App: " + nickname + " si è disconnesso.");
        }
    }

    // Riconessione al client "disconnesso" per permettere la disconnessione vera e propria
    messaggioServerOut = new PrintWriter(clientSocket.getOutputStream(), true);
```

La disconnessione vera e propria viene effettuata dopo la ricezione del comando QUIT.

```
// Disconnessione vera e propria
// Chiusura del socket di comunicazione
clientSocket.close();
// Chiusura dello stream dal client
messaggioClientIn.close();
// Chiusura dello stream verso il client
messaggioServerOut.close();
```